Daily Quant Digest: 2026-10-13 (b)

Today's topics

1. Probability & Statistics: Linearity of expectation and indicator-variable tricks (the noodle-loops problem)
2. Derivatives: US Treasuries: bills, notes and bonds, auctions, quoting in 32nds, accrued interest
3. Algorithms & Data Structures: Sliding window with two pointers ("Minimum Window Substring")

Difficulty today: Easy-Medium / Easy / Medium-Hard. The probability question is a famous one that looks hard and collapses with the right indicator; the algorithm is a step up from yesterday's prefix sums.

---

1. Probability & Statistics: Linearity of Expectation and Indicators

1.1 The statement, and why it is stronger than it looks

For any random variables $X_1, \dots, X_n$ with finite expectations, and constants a_i,

$$\mathbb{E}\Big[\sum_i a_i X_i\Big] = \sum_i a_i \, \mathbb{E}[X_i].$$

No independence is required. The X_i can be as tangled as you like. This is what makes it the single most useful tool in interview probability: you can compute the mean of a complicated count without ever finding its distribution.

Proof sketch (discrete case). Let ω range over outcomes. Then

$$\mathbb{E}[X+Y] = \sum_{\omega} (X(\omega)+Y(\omega))\,\mathbb{P}(\omega) = \sum_{\omega} X(\omega)\mathbb{P}(\omega) + \sum_{\omega} Y(\omega)\mathbb{P}(\omega).$$

That is it. Expectation is an integral, and integrals are linear. Dependence between X and Y lives in the *joint* distribution, but the sum above only ever weights each outcome by its own probability, so the joint structure never enters.

Contrast with variance, which *does* care about dependence:

$$\operatorname{Var}\Big(\sum_i X_i\Big) = \sum_i \operatorname{Var}(X_i) + 2\sum_{i<j} \operatorname{Cov}(X_i, X_j).$$

1.2 The indicator method

For an event A, the indicator $\mathbf{1}_A$ is 1 if A happens and 0 otherwise. Its key property: $\mathbb{E}[\mathbf{1}_A] = \mathbb{P}(A)$.

Recipe. To find $\mathbb{E}[N]$ where N counts "how many things of some kind happen":

1. Write $N = \sum_i \mathbf{1}_{A_i}$, one indicator per *potential* thing.
2. Then $\mathbb{E}[N] = \sum_i \mathbb{P}(A_i)$.
3. Usually symmetry makes all $\mathbb{P}(A_i)$ equal, so $\mathbb{E}[N] = (\text{number of terms}) \times p$.

The whole art is step 1: choosing what to index over. Classic examples you should be able to do in your head:

Quantity	Indicators	Answer
Fixed points of a random permutation of n	$\mathbf{1}\{\sigma(i) = i\}$, $p = 1/n$	1
Overlapping "HH" in n fair flips	$\mathbf{1}\{\text{flips } i, i+1 \text{ both H}\}$, $p = 1/4$	$(n-1)/4$
Runs in n fair flips	$1 + \sum_{i \geq 2} \mathbf{1}\{\text{flip } i \neq \text{flip } i-1\}$	$1 + (n-1)/2$
Records ("new highs") in i.i.d. continuous sequence	$\mathbf{1}\{X_k = \max(X_1..X_k)\}$, $p = 1/k$	$H_n = \sum_{k=1}^{n} 1/k$
Coupon collector, n types	$T = \sum_j G_j$, $G_j \sim \text{Geom}\left(\frac{n-j+1}{n}\right)$	nH_n

The records result is worth a trading interpretation: if daily P&L were i.i.d., the expected number of "new all-time high" days in 100 days would be $H_{100} \approx 5.19$. Records are rare, and they get rarer like $1/k$.

The coupon collector line uses linearity on waiting times rather than indicators: the time to go from $j-1$ to j distinct coupons is geometric with success probability $(n-j+1)/n$, mean $n/(n-j+1)$. Summing gives nH_n. For a die, $6H_6 = 14.7$ rolls to see every face.

1.3 Variance with indicators: pairs

If $N = \sum_i \mathbf{1}_{A_i}$, then $N^2 = \sum_i \mathbf{1}_{A_i} + \sum_{i \neq j} \mathbf{1}_{A_i \cap A_j}$ (since $\mathbf{1}^2 = \mathbf{1}$). So

$$\mathbb{E}[N^2] = \sum_i \mathbb{P}(A_i) + \sum_{i \neq j} \mathbb{P}(A_i \cap A_j).$$

You only need *pairwise* intersections, never the full distribution.

Example: fixed points. $\mathbb{P}(\sigma(i) = i, \sigma(j) = j) = \frac{1}{n(n-1)}$. So $\mathbb{E}[N^2] = 1 + n(n-1) \cdot \frac{1}{n(n-1)} = 2$, and $\text{Var}(N) = 2 - 1^2 = 1$. Mean 1, variance 1, for every $n \geq 2$: consistent with N converging to Poisson(1), and indeed $\mathbb{P}(N = 0) \to e^{-1}$ (the derangement result).

A more general version of the same idea: $\mathbb{E}\binom{N}{2} = \sum_{i<j} \mathbb{P}(A_i \cap A_j)$. Factorial moments are just "expected number of pairs", which is often the quickest route.

1.4 The tail-sum formula

For a non-negative integer random variable,

$$\mathbb{E}[X] = \sum_{k \geq 1} \mathbb{P}(X \geq k).$$

Proof. $X = \sum_{k \geq 1} \mathbf{1}\{X \geq k\}$ (a value of 3 is counted by $k = 1, 2, 3$). Take expectations; swapping sum and expectation is fine because the terms are non-negative (monotone convergence). The continuous analogue is $\mathbb{E}[X] = \int_0^\infty \mathbb{P}(X > x)\, dx$ for $X \geq 0$.

This is the right tool for **maxima and minima**, where you know CDFs but not much else. *Example:* max of two dice.

$$\mathbb{E}[M] = \sum_{k=1}^{6} \left(1 - \left(\tfrac{k-1}{6}\right)^2\right) = 6 - \frac{0 + 1 + 4 + 9 + 16 + 25}{36} = 6 - \frac{55}{36} = \frac{161}{36} \approx 4.47.$$

1.5 Pitfalls

- **Linearity does not pass through nonlinear functions.** $\mathbb{E}[\max(X, Y)] \neq \max(\mathbb{E}X, \mathbb{E}Y)$, $\mathbb{E}[1/X] \neq 1/\mathbb{E}[X]$ (Jensen says $\geq$ for $X > 0$), and $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$ needs uncorrelatedness. In option terms: $\mathbb{E}[(S-K)^+] \geq (\mathbb{E}S - K)^+$, and the gap is the time value. Option value *is* the failure of linearity through a convex payoff.
- **Infinite sums need care.** $\mathbb{E}[\sum_{i=1}^{\infty} X_i] = \sum_i \mathbb{E}[X_i]$ holds if $X_i \geq 0$, or if $\sum_i \mathbb{E}|X_i| < \infty$. Without one of those, it can fail (the classic doubling strategy: each round has expected gain 0, but the stopped total is +1 almost surely).
- **A random number of terms.** $\mathbb{E}[\sum_{i=1}^{N} X_i] = \mathbb{E}[N]\mathbb{E}[X]$ needs extra conditions on N (it is Wald's identity, coming later in the curriculum). Linearity alone does not give it.
- **Choosing the wrong indicator.** If your indicators overlap in a way that double-counts, the expectation is of the wrong thing. Always check: "does N really equal this sum, outcome by outcome?"
- **Expectation is not the typical value.** Your answer may be non-integer (3.28 loops) or the distribution may be very skewed. Interviewers often follow up with "what is the probability it equals the mean?" to test this.

1.6 Interview question

> A bowl contains 100 strands of spaghetti, so 200 loose ends. Repeatedly, you pick two loose ends uniformly at random (from all loose ends remaining) and tie them together. You stop when no loose ends remain, at which point everything has been tied into loops. What is the expected number of loops?

> Follow-ups: (a) approximately how large is this for large n? (b) what is the probability you end with a single giant loop? (c) what is the variance?

1.7 Solution

Step 1: understand the state. At any moment, the bowl contains some number k of open "strands" (each original noodle, or chain of noodles tied end to end, that is not yet closed). Each open strand has exactly 2 loose ends, so there are $2k$ loose ends. Closed loops have no loose ends and never matter again. Initially $k = 100$.

Step 2: what one tie does. Pick two loose ends. Equivalently (and this is the key simplification) pick the *first* end in any way you like, then the second uniformly among the other $2k - 1$ ends.

- With probability $\frac{1}{2k-1}$ the second end is the other end of the *same* strand. Tying them closes a loop. Open strands: $k \to k - 1$.
- Otherwise it belongs to a different strand. Tying joins two strands into one longer strand. Open strands: $k \to k - 1$.

Either way k drops by exactly one. So there are exactly $n = 100$ ties, and at the tie made when there are k open strands, a loop forms with probability $\frac{1}{2k-1}$, *regardless of what happened before*.

Step 3: indicators. Let $L_k = \mathbf{1}\{\text{a loop is formed at the tie made with } k \text{ open strands}\}$. Total loops $= \sum_{k=1}^{n} L_k$, so by linearity

$$\mathbb{E}[\text{loops}] = \sum_{k=1}^{n} \frac{1}{2k-1} = 1 + \frac{1}{3} + \frac{1}{5} + \cdots + \frac{1}{199}.$$

Step 4: evaluate. Numerically, for $n = 100$ this is ≈ 3.284.

Sanity checks.

- $n = 1$: one noodle, one tie, one loop. Formula: 1. Correct.
- $n = 2$: four ends. The first end's partner is one of 3; with probability $1/3$ it is its own other end, giving a loop and then a second loop from the remaining noodle (2 loops). Otherwise we get 1 big strand then 1 loop. $\mathbb{E} = \frac{1}{3} \cdot 2 + \frac{2}{3} \cdot 1 = \frac{4}{3}$. Formula: $1 + \frac{1}{3} = \frac{4}{3}$. Correct.
- $n = 3$: formula gives $1 + \frac{1}{3} + \frac{1}{5} = \frac{23}{15} \approx 1.533$; a 100,000-trial simulation gave 1.535.

(a) Asymptotics (the interview trick). Write odd reciprocals as "all reciprocals minus even ones":

$$\sum_{k=1}^{n} \frac{1}{2k-1} = H_{2n} - \tfrac{1}{2} H_n \approx \big(\ln 2n + \gamma\big) - \tfrac{1}{2}\big(\ln n + \gamma\big) = \tfrac{1}{2}\ln n + \ln 2 + \tfrac{\gamma}{2},$$

with $\gamma \approx 0.5772$. For $n = 100$: $\frac{1}{2}(4.605) + 0.693 + 0.289 = 3.284$. Matches to 3 decimals. The key takeaway: loops grow like $\frac{1}{2}\ln n$, so even a million noodles give only about 8.2 loops.

(b) One giant loop. You need *no* loop until the very last tie (with $k = 1$ a loop is forced). The events are independent across ties (each probability depends only on k), so

$$\mathbb{P}(\text{one loop}) = \prod_{k=2}^{n}\left(1 - \frac{1}{2k-1}\right) = \prod_{k=2}^{n}\frac{2k-2}{2k-1}.$$

For $n = 3$: $\frac{2}{3}\cdot\frac{4}{5} = \frac{8}{15}$ (simulation: 0.533). For $n = 100$ it is ≈ 0.0887, and a Wallis-product approximation gives $\approx \frac{\sqrt{\pi}}{2\sqrt{n}} = 0.0886$.

(c) Variance. Because the L_k are independent Bernoullis,

$$\mathrm{Var} = \sum_{k=1}^{n}\frac{1}{2k-1}\left(1 - \frac{1}{2k-1}\right) \approx 2.05 \quad (n = 100),$$

standard deviation about 1.43. Note that independence was a bonus here: linearity gave the mean without it.

Why this problem is a good test. Most candidates try to track the lengths of strands, which is a nightmare. The insight is that only the *number* of open strands matters for the loop probability, and that count falls deterministically by one per tie.

---

2. Derivatives: US Treasuries (Bills, Notes, Bonds, Quoting, Accrued Interest)

Treasuries are the risk-free benchmark for USD rates: the collateral in repo, the hedge for swaps and mortgages, the deliverable into Treasury futures (tomorrow's natural next topic). Every rates and FX desk touches them. Getting the conventions exactly right is a classic screening test.

2.1 The instruments

Type	Original maturities	Coupon	Quoted as
Bills	4, 6, 8, 13, 17, 26, 52 weeks (plus cash-management bills)	None (zero coupon, sold at a discount)	Discount rate, actual/360
Notes	2, 3, 5, 7, 10 years	Fixed, semiannual	Price in 32nds
Bonds	20, 30 years	Fixed, semiannual	Price in 32nds
TIPS	5, 10, 30 years	Real coupon on CPI-indexed principal	Real price/yield
FRNs	2 years	Floats off the 13-week bill rate	Discount margin

Who trades them: central banks and reserve managers (huge, mostly buy-and-hold), primary dealers (obliged to bid at auctions and make markets), asset managers matching benchmarks, hedge funds (relative value, basis trades), banks managing liquidity (HQLA requirements), and everyone hedging duration.

2.2 Auctions

Treasury sells new issues at regular, pre-announced auctions. Mechanics:

- **Announcement** a few days before, giving size and settlement date. The security trades **when-issued (WI)** from announcement until issue, quoted in yield.
- **Bids**: *non-competitive* bidders (small, retail) accept whatever yield results; *competitive* bidders submit a yield and amount.
- **Single-price (uniform) format**: bids are filled from the lowest yield upward until the size is covered. Everyone accepted gets the **highest accepted yield** (“stop-out” or “high yield”). For a coupon security the coupon is set to the high yield rounded *down* to the nearest 1/8%, so the issue price is at or slightly below par.
- **Results** that the street watches: the **tail** (high yield minus the WI yield at the 1:00pm New York deadline; a positive tail is a weak auction, a “stop-through” is strong), the **bid-to-cover** ratio, and the share taken by **indirect** (often foreign official), **direct**, and **primary dealer** bidders. A large dealer take means the street had to absorb supply it did not want.

The most recently auctioned security in each tenor is **on-the-run (OTR)**; older ones are **off-the-run**. OTRs are more liquid and often trade **special** in repo (you can borrow cash very cheaply against them), so they trade at a lower yield (“rich”) than nearby off-the-runs. That spread is the OTR premium, and trading it was LTCM’s famous trade.

2.3 Quoting coupons in 32nds

Note and bond prices are per 100 face, with the fractional part in **32nds**:

- `99-16` means $99 + \frac{16}{32} = 99.5$.
- `99-16+` means $99 + \frac{16.5}{32} = 99.515625$ (the "+" is half a 32nd, i.e. a 64th).
- `99-162` means $99 + \frac{16+2/8}{32} = 99.5078125$. The third digit is in eighths of a 32nd (2 = quarter, 4 = half, 6 = three quarters). So `99-164` is the same as `99-16+`.

Value of a tick. On \$10M face, one 32nd is $10{,}000{,}000 \times \frac{1}{100} \times \frac{1}{32} = \$3{,}125$. A "plus" (64th) is \$1,562.50. Memorise: **one 32nd on \$1M is \$312.50**.

Never confuse `99-16` with `99.16`. That error is \$1,000+ per million and a classic junior blunder.

2.4 Clean price, dirty price and accrued interest

Quoted prices are **clean**. What you pay is the **dirty** (invoice, full) price:

$$\text{Dirty} = \text{Clean} + \text{AI}, \qquad \text{AI} = \frac{C}{2} \times \frac{\text{days from last coupon to settlement}}{\text{days in the coupon period}}.$$

Treasuries use **actual/actual (ICMA)**: both day counts are actual calendar days, and the period length varies (181 to 184 days). Settlement is normally **T+1**, over Fedwire, delivery versus payment.

Why quote clean? The dirty price drifts up as coupon accrues and then drops by the full coupon on the payment date: a sawtooth. Stripping out accrued leaves a price that only moves when yields move, which is what you want to see on a screen. The seller is still compensated for the coupon they earned while holding the bond: the buyer pays AI and then receives the full coupon.

2.5 Pricing from yield

With w = (days from settlement to next coupon) / (days in current period) and N remaining coupons, the street-convention yield y solves

$$\text{Dirty} = \sum_{j=0}^{N-1} \frac{C/2}{(1 + y/2)^{j+w}} + \frac{100}{(1 + y/2)^{N-1+w}}.$$

Yields are semiannually compounded, which matters when comparing to USD SOFR swap rates (annual fixed leg, actual/360) or to annually compounded EUR government yields.

DV01 is the dollar change in dirty price for a 1bp fall in yield. **Modified duration** $= \text{DV01}/\text{Dirty} \times 10^4$.

2.6 Worked scenario 1: buying a note between coupon dates

On Tuesday 13 Oct 2026 you buy **\$10M face** of a $4\frac{1}{8}\%$ note maturing **15 Nov 2032**, at `99-16+`. Settlement is T+1: **Wed 14 Oct 2026**. Coupons are paid 15 May and 15 Nov.

Step 1: clean price. $99 + 16.5/32 = 99.515625$.

Step 2: accrued interest.
- Last coupon 15 May 2026, next 15 Nov 2026. Period length = 184 days.
- Days accrued (15 May to 14 Oct) = 152 days.
- $\text{AI} = \frac{4.125}{2} \times \frac{152}{184} = 2.0625 \times 0.826087 = 1.703804$ per 100.

Step 3: invoice.
- Dirty $= 99.515625 + 1.703804 = 101.219429$.
- Cash paid $= 101.219429\% \times 10{,}000{,}000 = \$10{,}121{,}942.93$, of which \$170,380.43 is accrued.

Step 4: yield. $w = 32/184$ (14 Oct to 15 Nov is 32 days), $N = 13$ coupons (Nov 2026 to Nov 2032). Solving numerically gives $y \approx 4.2155\%$. Sanity check: price slightly below par, so yield slightly above the 4.125% coupon. Good.

Step 5: risk. DV01 ≈ 0.05306 per 100 face, i.e. **\$5,306 per bp on \$10M**. Modified duration ≈ 5.24. In ticks: 1bp ≈ 1.70 32nds.

Step 6: a shock. Yields jump 5bp (say a hot CPI print) before you do anything. Repricing at $y = 4.2655\%$: dirty falls by 0.26487, so clean ≈ 99.2508, which the screen shows as about `99-08`. P&L $\approx -\$26{,}487$. The DV01 approximation gives $5 \times 5{,}306 = \$26{,}528$; the small difference is convexity working in your favour.

2.7 Worked scenario 2: a 26-week bill

The 26-week bill (182 days to maturity) is quoted at a **discount rate** $d = 3.80\%$.

- Price $= 100\big(1 - d \cdot \frac{182}{360}\big) = 100(1 - 0.019211) = 98.078889$.
- On \$10M face you pay \$9,807,888.89 and receive \$10M at maturity: \$192,111.11 interest.
- **Money-market yield** (actual/360, on price not face): $\frac{360\,d}{360 - d \cdot 182} = 3.874\%$.
- **Bond-equivalent yield** (for bills up to half a year): $\frac{365\,d}{360 - d \cdot 182} = 3.928\%$. Check: $(100/98.078889 - 1) \times 365/182 = 3.928\%$.

The discount rate always understates the true yield: it divides by face (not price) and uses 360 days. To compare a bill to a coupon note, convert to BEY.

2.8 What can go wrong

- **Convention errors:** 32nds vs decimals; using clean price in a PV formula; wrong period length in accrued; forgetting that a coupon date falling on a weekend pays on the next business day but accrual is unadjusted.
- **Repo and specialness:** a long financed in repo earns coupon accrual minus repo cost (carry). If you are short a bond that goes special, borrowing it to deliver becomes very expensive.

- **Settlement fails:** if you cannot deliver, you are charged a fails penalty (the TMPG fails charge, since 2009: 3% minus the lower bound of the fed funds target range, floored at zero, annualised), so a failed delivery is never free.
- **Liquidity events:** March 2020 showed that even Treasuries can gap when dealers' balance sheets are full and basis trades unwind together.
- **Curve risk:** a DV01-neutral hedge with a different maturity still leaves you exposed to steepening or flattening.

2.9 Likely interview questions

1. **"What is one 32nd worth on $50M of a 10-year?"** $50{,}000{,}000/100/32 = \$15{,}625$. (Price ticks are independent of maturity; the *yield* move per tick is not.)
2. **"Why are bonds quoted clean? Who gets the coupon if I sell the day before payment?"** Clean removes the accrual sawtooth. The buyer receives the full coupon, but has paid you accrued for all but one day in the invoice price, so economically you get your share.
3. **"Why does the on-the-run 10-year yield less than an off-the-run with nearly the same maturity?"** Liquidity premium and repo specialness: holders can finance it below general collateral, so they accept a lower yield. The spread tends to widen in stress and compress when a new issue takes over as OTR.

---

3. Algorithms & Data Structures: Sliding Window ("Minimum Window Substring")

3.1 Problem

Given strings `s` and `t`, return the shortest contiguous substring of `s` that contains every character of `t`, **including duplicates** (if `t = "AAB"`, the window needs at least two A's and one B). If no such window exists, return `""`. If several shortest windows exist, return the leftmost.

Trading framing: `s` is the sequence of venues that filled your child orders through the day, and `t` is a basket you need; find the shortest time span in which every required fill appeared.

Constraints: $1 \le |s|, |t| \le 10^5$, ASCII characters.

Examples:

s	t	Output
`"ADOBECODEBANC"`	`"ABC"`	`"BANC"`
`"a"`	`"a"`	`"a"`
`"a"`	`"aa"`	`""` (only one `a` available)
`"BBAACB"`	`"ABB"`	`"BBA"`

3.2 Brute force

For each start `i`, extend `j` until the window `s[i..j]` covers `t`, then record its length and move on to the next `i` (extending further only makes it longer). With a 128-slot count array, checking coverage is $O(128)$, so this is $O(n^2)$ time overall. At $n = 10^5$ that is 10^{10} operations: too slow.

3.3 The insight: windows are monotone

Two observations:

1. **If `s[i..j]` covers `t`, then so does every larger window `s[i'..j']` with $i' \le i$, $j' \ge j$.** Adding characters never removes coverage.
2. Therefore, as the right end `j` moves right, the *smallest valid left end* `i(j)` never moves left. If it did, then a longer window would have become invalid, contradicting (1).

So both pointers only ever move forward. That is the sliding-window (two-pointer) pattern: **expand right until valid, then shrink left while still valid, recording the best**. Each index enters the window once and leaves at most once, so total work is $O(n)$.

The remaining question is how to test "is the window valid?" in $O(1)$ rather than $O(128)$. Keep:

- `need[c]`: how many more copies of `c` the window still needs (it can go negative, meaning surplus).
- `missing`: the total number of required characters not yet covered, $\sum_c \max(\text{need}[c], 0)$.

The window is valid exactly when `missing == 0`. Updating:

- Adding char `c` on the right: if `need[c] > 0` it was genuinely needed, so `missing--`. Then `need[c]--`.
- Removing char `c` on the left: `need[c]++`. If `need[c]` is now positive, we just removed a copy that was required, so `missing++`.

Trace on `s = "ADOBECODEBANC"`, `t = "ABC"`. Right pointer reaches index 5 (`C`): window `ADOBEC` (length 6) is the first valid one. Removing `A` breaks it. The right pointer runs to index 10 (`A`), window `DOBECODEBA` is valid; shrinking removes D, O, B (the surplus B at index

3 is fine since another B is at 9), E, then C breaks it. Lengths seen were 10 down to 6, none better than 6. At index 12 (`C`) the window `ODEBANC` is valid; shrinking drops O, D, E to reach `BANC` (length 4), then removing B breaks it. Answer: `BANC`.

Complexity: $O(|s| + |t|)$ time, $O(1)$ extra space (a fixed array of 128 ints; $O(\Sigma)$ for a general alphabet).

3.4 C++ solution

```cpp
#include <array>
#include <string>

std::string minWindow(const std::string& s, const std::string& t) {
    if (t.empty() || s.size() < t.size()) return "";

    std::array<int, 128> need{};          // all zeros
    for (unsigned char c : t) ++need[c];

    int missing = static_cast<int>(t.size());
    int bestStart = 0;
    int bestLen = -1;                     // -1 = nothing found yet

    int left = 0;
    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        unsigned char in = s[right];
        if (need[in] > 0) --missing;
        --need[in];

        while (missing == 0) {            // window [left, right] is valid
            int len = right - left + 1;
            if (bestLen == -1 || len < bestLen) {
                bestLen = len;
                bestStart = left;
            }
            unsigned char out = s[left];
            ++need[out];
            if (need[out] > 0) ++missing; // removed a required copy
            ++left;
        }
    }
    return bestLen == -1 ? "" : s.substr(bestStart, bestLen);
}
```

The full file `2026-10-13-b.cpp` adds a `main` that checks six fixed cases and cross-checks 20,000 random small inputs against the brute force. All pass with `g++ -std=c++17 -O2 -Wall`.

3.5 Walkthrough of the C++

- **`const std::string& s`**: pass by reference to const. No copy of a 100,000-character string is made, and the compiler stops us modifying it. For any non-trivial object you only read, this is the default parameter style.
- **`std::array<int, 128> need{};`**: a fixed-size array living on the stack, with value semantics (it copies like an `int`, unlike a C array that decays to a pointer). The empty braces `{}` *value-initialise* it, so every element is 0. Writing `std::array<int,128> need;`

without braces would leave the ints uninitialised: a real and common bug. Compared with `std::unordered_map<char,int>`, the array has no hashing and no heap allocation, which is much faster and very cache friendly.

- **`for (unsigned char c : t)`**: a range-based for loop over the characters. Declaring the loop variable as `unsigned char` matters: plain `char` may be signed, and a byte above 127 would become a negative index. Casting to `unsigned char` before indexing is the standard defensive habit.
- **`static_cast<int>(t.size())`**: `size()` returns `std::size_t` (unsigned). Mixing signed and unsigned in comparisons triggers `-Wall` warnings and can bite you (`0u - 1` is a huge number). Converting once, explicitly, keeps loop arithmetic in `int`.
- **`++need[c]` vs `need[c]++`**: for an `int` they compile to the same thing here, but prefix increment is the idiomatic default because for iterators and other class types the postfix form must make a copy.
- **The `while (missing == 0)` inside the `for`**: this nested loop looks quadratic but is not, because `left` only increases and is bounded by `right + 1`. Across the whole run it executes at most $|s|$ times. This "amortised" argument is how you justify every two-pointer solution.
- **`bestLen == -1 ? "" : s.substr(bestStart, bestLen)`**: the conditional operator; both branches convert to `std::string`. `substr(pos, count)` returns a new string, so it is called once at the end rather than on every improvement (copying on every improvement would cost $O(n)$ per copy).
- **In the test harness**: `struct Case { std::string s, t, expected; };` with a braced initializer list builds the test table; `std::mt19937` with a fixed seed and `std::uniform_int_distribution` give reproducible random tests; `assert` from `<cassert>` aborts on a failed check (note it is disabled if `NDEBUG` is defined).

3.6 Variations you should be ready for

- **Longest substring with at most K distinct characters**: same expand/shrink skeleton, but you maximise length and shrink while the window is *invalid*.
- **Shortest subarray with sum $\geq$ target (all positives)**: same template with a running sum. With negative numbers the monotonicity in 3.3 breaks, and you need prefix sums plus a monotonic deque (coming in the stacks topic).
- **Find all anagrams of `t` in `s`**: a fixed-size window of length $|t|$ sliding by one, with the same `need`/`missing` bookkeeping.
- **Rule of thumb:** sliding window works when validity is monotone in window size. If adding an element can both help and hurt (negative numbers, "exactly K" constraints), either convert the problem (e.g. exactly K = at most K minus at most $K-1$) or switch technique.